

SILVER SUITS AI

AI Voice RM Platform

Technical Architecture

Integrated: Demo Codebase + Production Target + Roadmap

Client: Indian Bank

Provider: Silver Suits AI

Version: 3.0 Integrated — May 2026

Repo: Voice-to-Command (demo codebase)

Status: Confidential — Internal Use Only

Table of Contents

1. Executive Summary & Scope
2. Demo Codebase
3. System Architecture — Current Demo (Figures 1–4)
4. System Architecture — Production Target (Figures 5–8)
5. Complete Data Workflow (Demo → Production)
6. Behavior Analysis & AI Triggers
7. Rule Engine — Demo vs Production
8. AG-UI Streaming Agents
9. Voice Bot Interaction & Logging
10. Bank DB Integration (Productization V1+)
11. Analytics Dashboard (Productization V2/V3)
12. Resource Requirements
13. Timeline & Version Feature List
14. Compliance Quick-Reference
15. Exception Handling & Fallbacks
16. Non-Functional Requirements

List of Figures

Figure	Title	Section
1	Current demo — layered architecture (Voice/UI + AG-UI)	§3
2	Voice turn sequence (demo)	§3
3	AG-UI SSE screen agent flow (demo)	§3
4	Dialogue state machine + repository map (demo)	§3
5	Production — six-layer architecture	§4
6	Production runtime — voice + rule engine	§4
7	Rule engine pipeline (production)	§4 / §7
8	DocumentDB + AWS deployment (production)	§4

1. Executive Summary & Scope

The AI Voice RM Platform is a web-based AI banking assistant embedded inside the Indian Bank mobile app. When a user shows frustration signals rage-clicking, repeated invalid field entries, or extended inactivity the system automatically presents a contextual AI assistant popup. The user can speak or type; the AI guides them through the form, executes banking transactions, and routes them to the right banking journey, all within the mobile app session.

The demo codebase (Voice-to-Command repo) is a fully working web simulation of this product. It runs a React frontend (simulated mobile banking UI) backed by an Express server with ElevenLabs STT, OpenAI LLM, and Cartesia TTS — all interaction stays inside the bank app. The productization roadmap adds bank DB integration, analytics, rule engine, and a visual rule-builder.

Metric	Demo (Codebase Today)	Full Product (V3)
Trigger mechanism	Rage-tap detection (JS hook) or DemoPanel button	Rage taps + inactivity + bank API events
Interaction channel	In-app popup (web session only)	In-app popup + push notification (V2)
STT	ElevenLabs Scribe v2 (proxy) or Web Speech API (browser)	ElevenLabs Scribe v2 (India data residency)
LLM	OpenAI GPT-4o-mini	Azure OpenAI GPT-4o (India endpoint)
TTS	Cartesia sonic-3 (proxy)	Cartesia sonic-3 (India region)
Banking flows	5 voice flows + 5 AG-UI screen agents	Same + extensible via new manifests
Data storage	In-memory mock only (no real DB writes)	Bank-owned DocumentDB (AWS Mumbai / OnPrem)

2. Demo Codebase

2.1 Repository Structure

Path	What It Is
client/src/App.jsx	Root React app — wires all state, voice hooks, and screens together
client/src/components/VoiceModal.jsx	Main AI assistant popup — chat bubbles, mic button, TTS playback, input bar
client/src/components/HomeScreen.jsx	Home banking screen — balance card, quick actions, AI concierge button
client/src/components/LoanApplicationScreen.jsx	Loan form with live AG-UI form-fill
client/src/components/ImpsFundTransferScreen.jsx	IMPS fund transfer form with AI guidance
client/src/components/CreateDepositScreen.jsx	FD/RD/MMD deposit creation with AI
client/src/components/TransactionHistoryScreen.jsx	Account statement with AI query support
client/src/components/RMHelpPrompt.jsx	Bottom-sheet popup shown on frustration detection
client/src/components/MpinSheet.jsx	MPIN authentication gate before payment confirmation
client/src/hooks/useRageDetect.js	Behavior detection: rage taps (5+ in 900ms) and repeated invalid fields
client/src/hooks/useElevenSpeech.js	ElevenLabs STT hook — MediaRecorder + VAD silence detection
client/src/hooks/useSpeech.js	Web Speech API fallback STT hook (browser built-in)
client/src/lib/cartesiaTts.js	Cartesia TTS client — single audio channel, VAD gate

client/src/engine/simEngine.js	Thin client wrapper — forwards all turns to POST /api/engine/turn
client/src/services/engineClient.js	HTTP client for engine API, account statement, and server reset
server/index.js	Express app — all API routes: /api/stt, /api/tts, /api/engine/*, /api/agui/*
server/engine/engine.js	Server-side state machine — processInput(), intent routing, state transitions
server/engine/saga.js	Generic saga runner — walks manifest steps, handles tool calls, validates
server/engine/llm.js	LLM call — OpenAI extract() for intent + slots + reply
server/engine/tools.js	Tool registry: contacts.search, billers.search, balance.check, payment execution
server/manifests/*.json	5 flow manifests: send_money, check_balance, internal_transfer, pay_bill, book_flight
server/manifestRegistry.js	Loads and indexes all manifests at startup
server/agui/loanAguiRoute.js	SSE route dispatcher for all 5 AG-UI agents
server/agui/homeAguiRunner.js	Home screen agent — navigate_to tool, UPI/IMPS routing rules
server/agui/loanAguiRunner.js	Loan LOS agent — set_field tool calls, form validation
server/agui/impsAguiRunner.js	IMPS transfer agent — fill transfer form fields
server/agui/depositAguiRunner.js	Deposit creation agent
server/agui/txnHistoryAguiRunner.js	Transaction history agent
server/data/backend.js	Pure mock banking logic: payments, transfers, bill pay, flight booking
server/data/mock.js	Static data: contacts, accounts, billers, per-transaction limits

2.2 Current Technical Stack

Component	Actual Implementation	File
STT (primary)	ElevenLabs Scribe v2 — MediaRecorder + VAD → POST /api/stt → ElevenLabs API	useElevenSpeech.js, server/index.js
STT (fallback)	Web Speech API (browser built-in) — Chrome/Edge best support	useSpeech.js
LLM	OpenAI GPT-4o-mini — intent + slots + reply extraction per turn	server/engine/llm.js
TTS	Cartesia sonic-3 — POST /api/tts → MP3 bytes → browser Audio()	cartesiaTts.js, server/index.js
State machine	Manifest/saga runner — 5 registered flows, 9 states	server/engine/engine.js, saga.js
AG-UI agents	5 SSE streaming agents via OpenAI function calling	server/agui/*Runner.js
Behavior detect	useRageDetect hook — pointerdown timestamps + invalid field counter	useRageDetect.js
Auth gate	MpinSheet — intercepts voice/tap confirm before executing payment	MpinSheet.jsx, App.jsx
Framework	React 18 + Framer Motion (animations) + Tailwind CSS	client/
Server	Node.js 20 LTS + Express 4 + cors + pino logging	server/

3. System Architecture — Current Demo (Voice-to-Command)

IN DEMO CODEBASE Figures 1–4 describe the verified implementation in the repository. Web in-app only — mock backend (no bank DB in demo).

TECHNICAL ARCHITECHTURE STILL TO BE DONE

3.1 Layered Architecture — Demo

Figure 1 shows five layers with two parallel paths: Voice/UIP (left) uses the manifest-driven state machine; AG-UI screen agents (right) use SSE streaming independently.

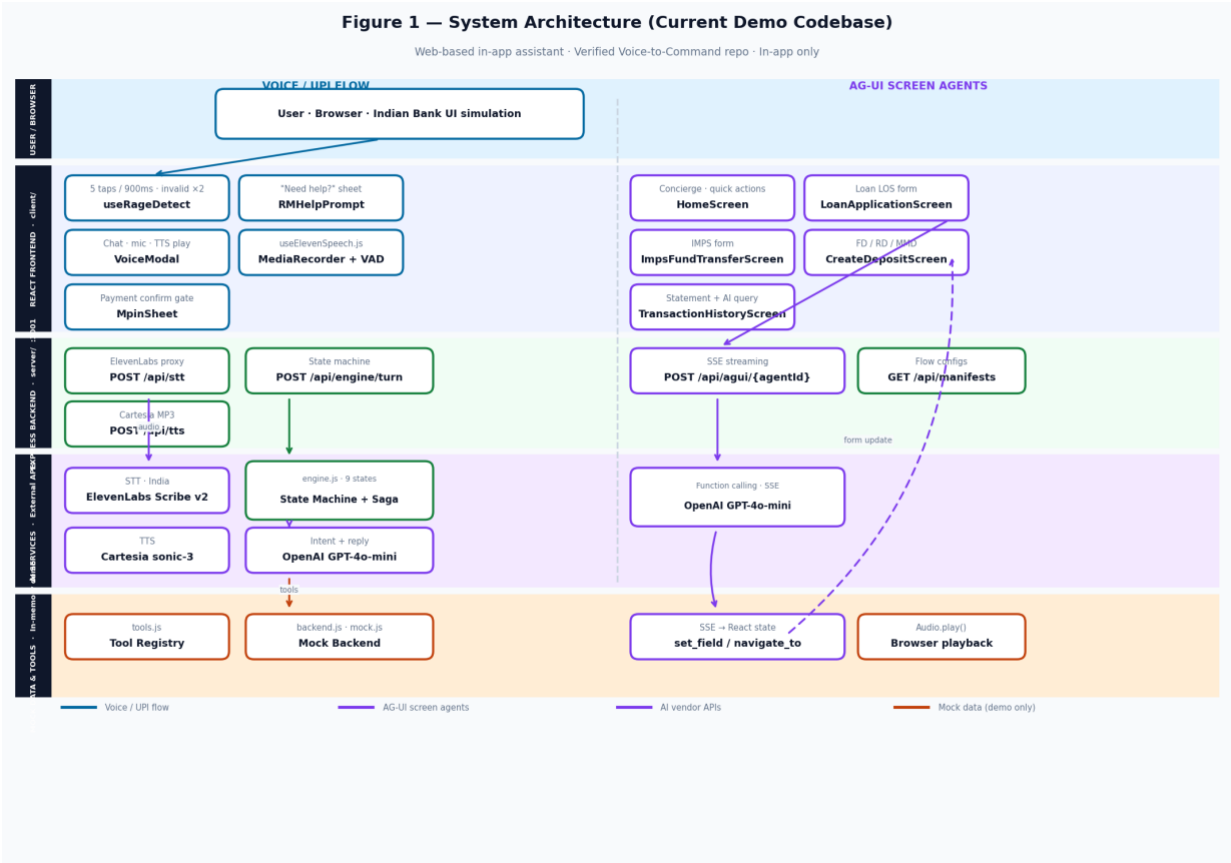


Figure 1 — Current demo: layered architecture (Voice/UIP + AG-UI, mock backend)

Layer	Components	Repo path
User / Browser	Indian Bank UI simulation	client/ · index.html
React Frontend	VoiceModal, rage detect, 5 banking screens	client/src/components/, hooks/
Express Backend	/api/stt, /api/tts, /api/engine/turn, /api/agui/*	server/index.js
AI Services	ElevenLabs STT, OpenAI GPT-4o-mini, Cartesia TTS	Proxied via server

Mock & Tools	Tool registry, in-memory contacts/billers/payments	server/data/, engine/tools.js
--------------	---	-------------------------------

3.2 Voice / UPI Flow — Sequence

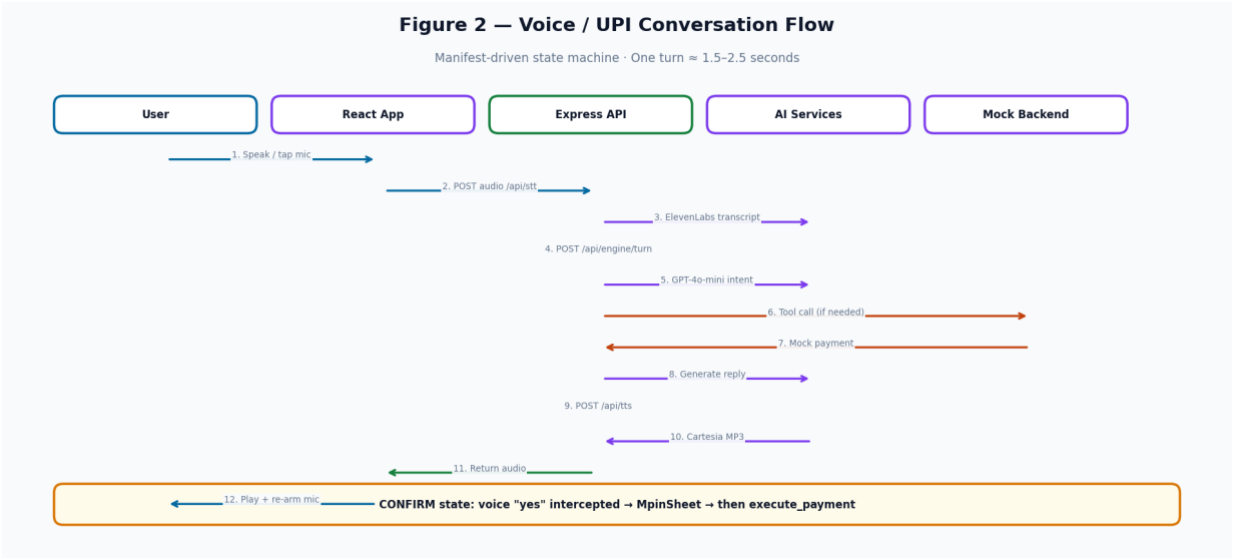


Figure 2 — Voice turn sequence: speak → STT → engine → tools → TTS → playback

3.3 AG-UI Screen Agent Flow — Sequence

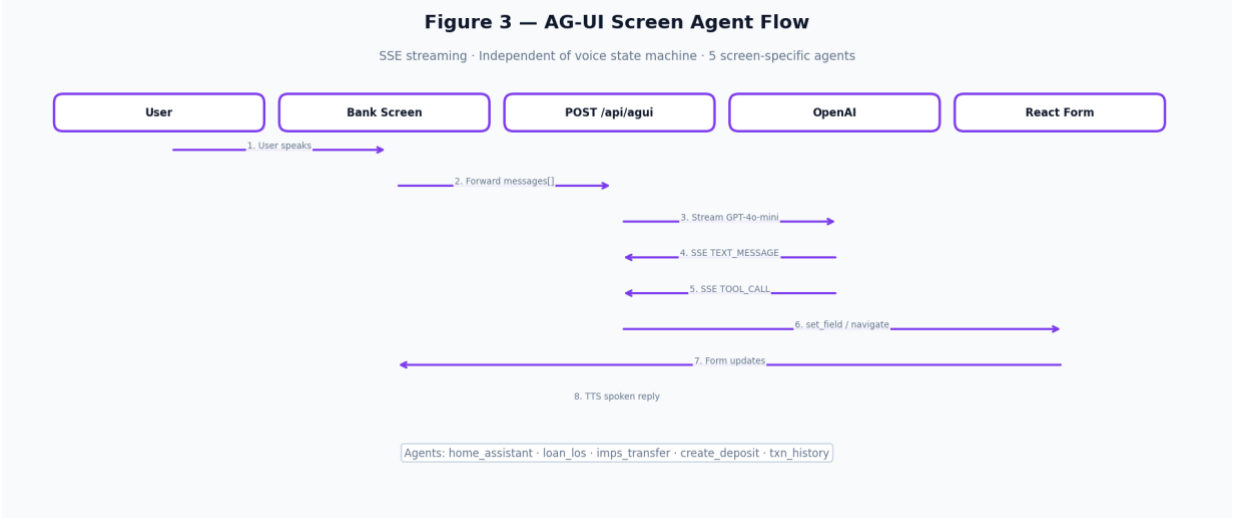


Figure 3 — AG-UI: screen → SSE agent → set_field / navigate_to → live form update

3.4 State Machine & Repository Map

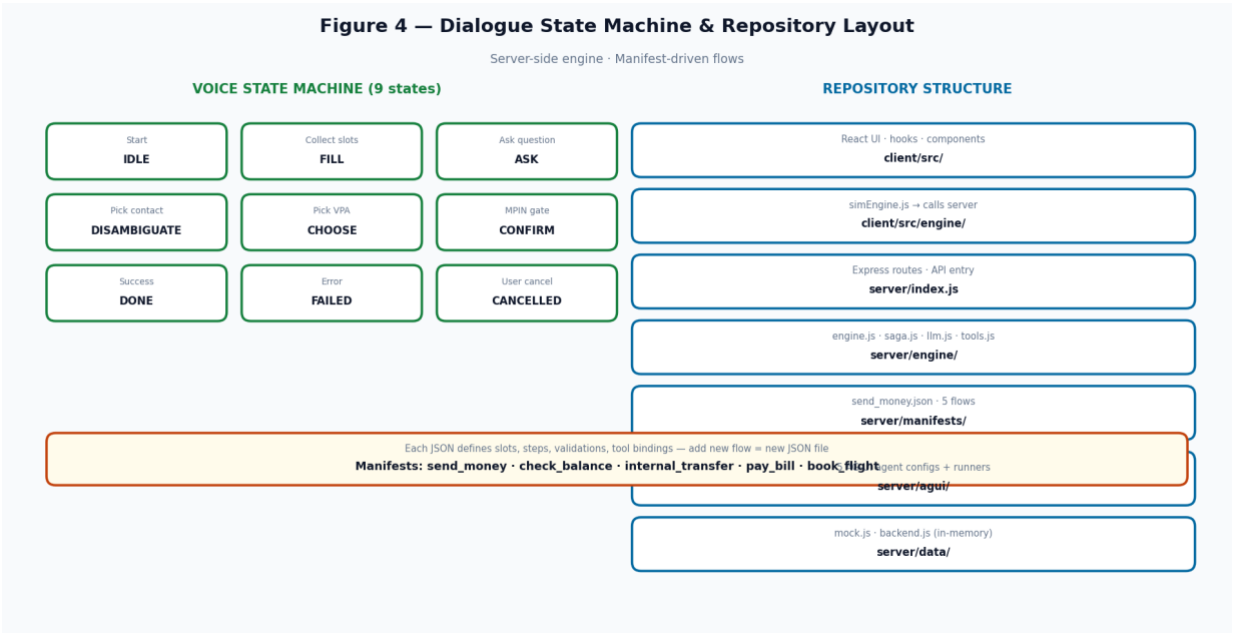


Figure 4 — Nine dialogue states + key repository folders

Manifest	Slots	Mock tools
send_money	recipient, amount, VPA	contacts.search → execute_upi_payment
check_balance	account type	balance.check
internal_transfer	amount, from, to	accounts.list → execute_transfer
pay_bill	biller, account, amount	billers.search → execute_bill_payment

4. System Architecture — Production Target

NOTE Figures 5–8 describe the V1+ product deployed in bank VPC (AWS Mumbai). Builds on the demo engine; adds bank integration, rule engine, DocumentDB, analytics.

4.1 Layered Production Architecture

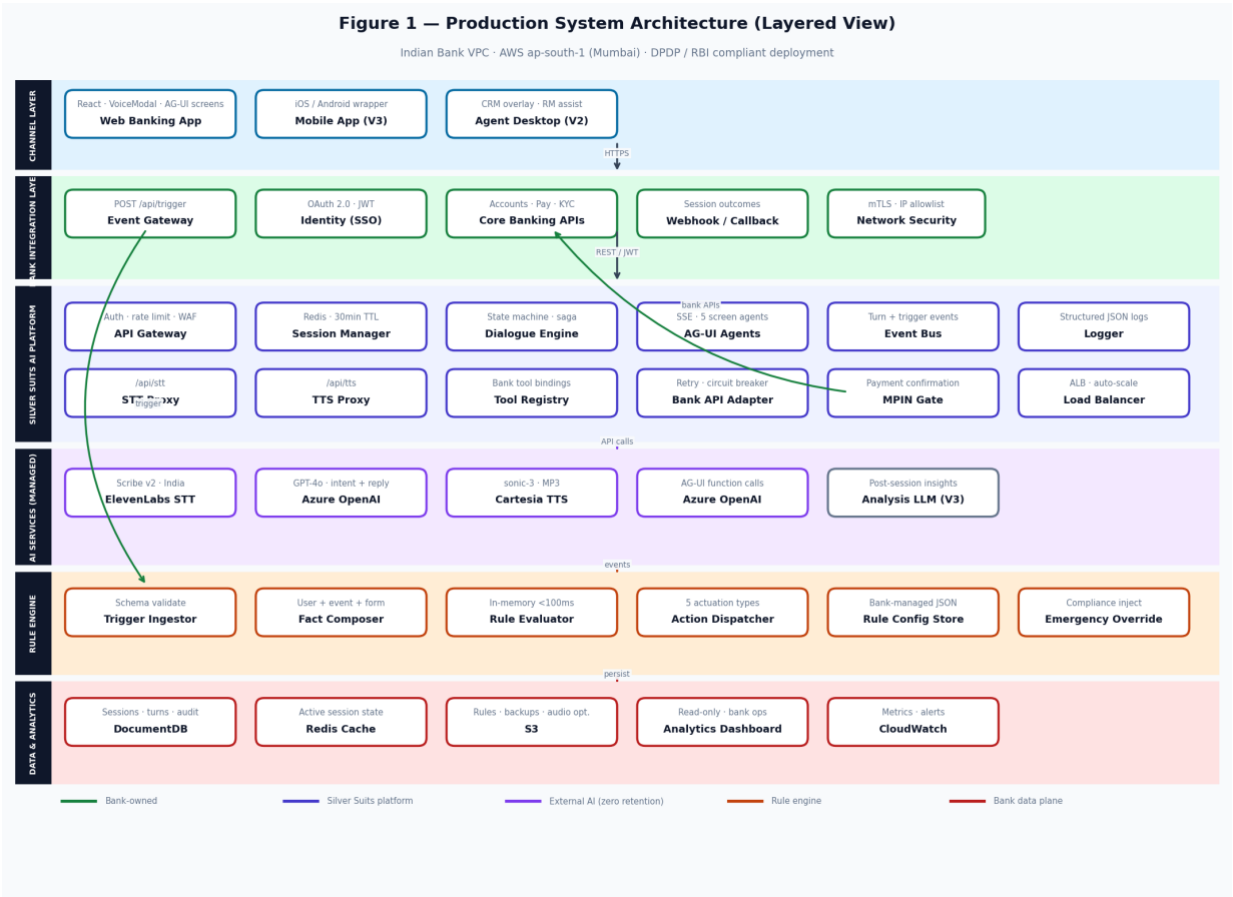


Figure 5 — Production: six layers (channel → bank → platform → AI → rules → data)

Layer	Owner	Key addition vs demo
Channel	Bank	Native mobile (V3), agent desktop (V2)
Bank Integration	Bank	SSO, CBS events, core APIs, webhooks, mTLS
Silver Suits Platform	Silver Suits	Gateway, Redis sessions, event bus, bank API adapter
AI Services	Silver Suits	Azure OpenAI India; zero retention contracts
Rule Engine	Silver Suits · bank configures	POST /api/trigger, JSON rules, <100ms

4.2 Runtime Flows — Voice + Rule Engine

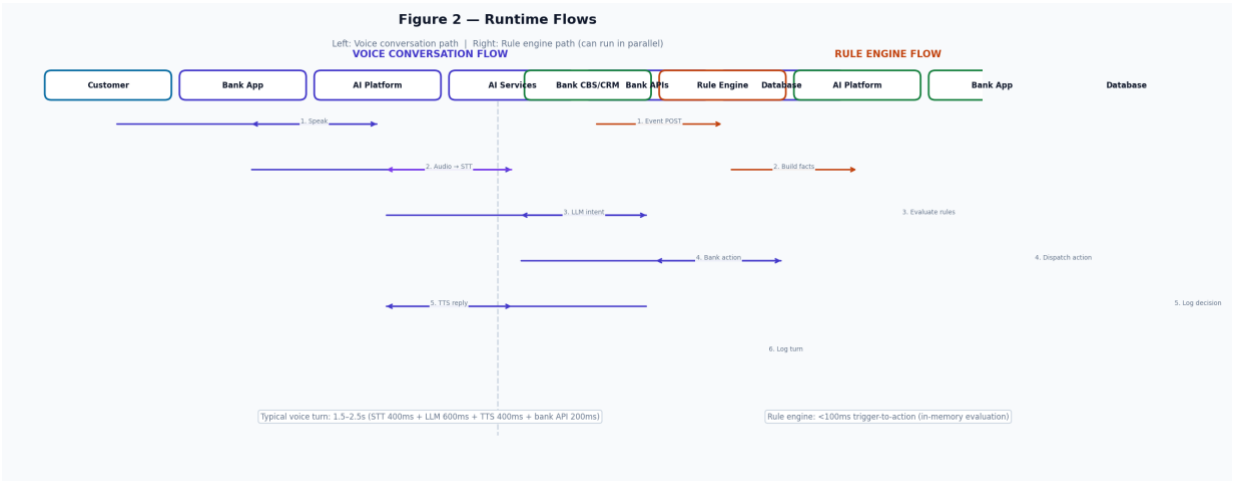


Figure 6 — Production runtime: voice turn path and rule engine path (parallel)

4.3 Rule Engine Architecture

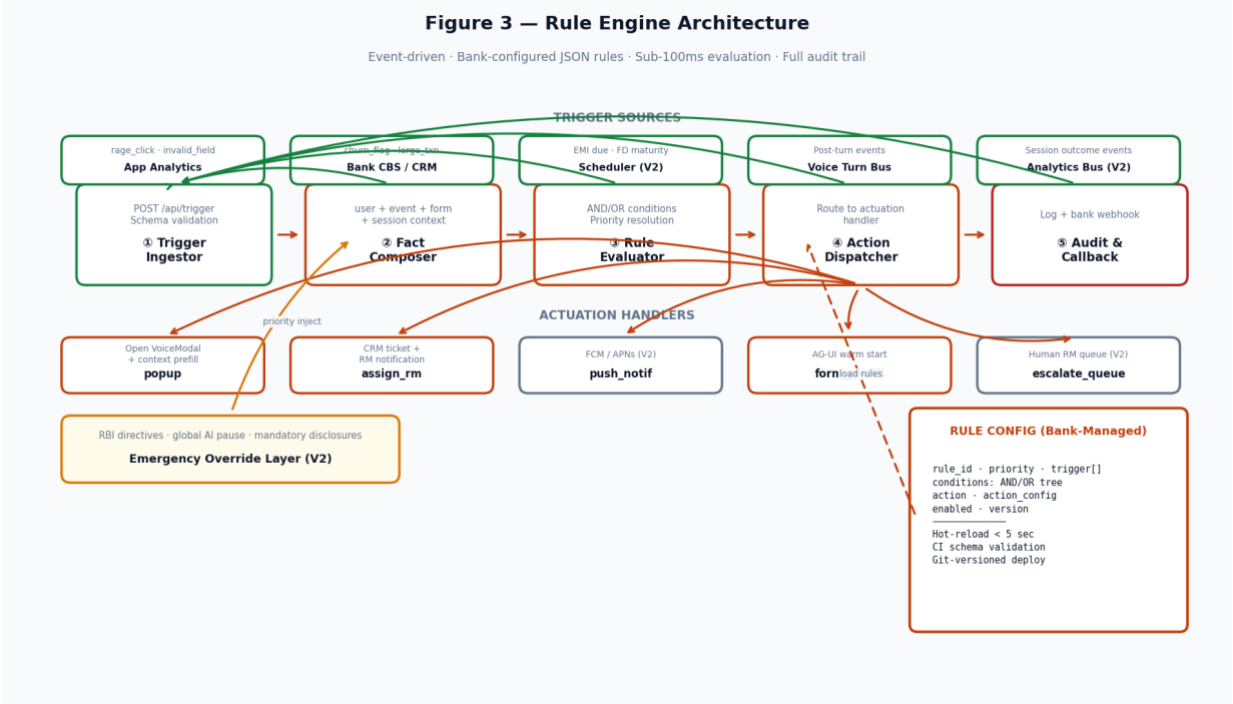


Figure 7 — Rule engine: trigger → facts → evaluate → dispatch → audit

4.4 Data Model & AWS Deployment

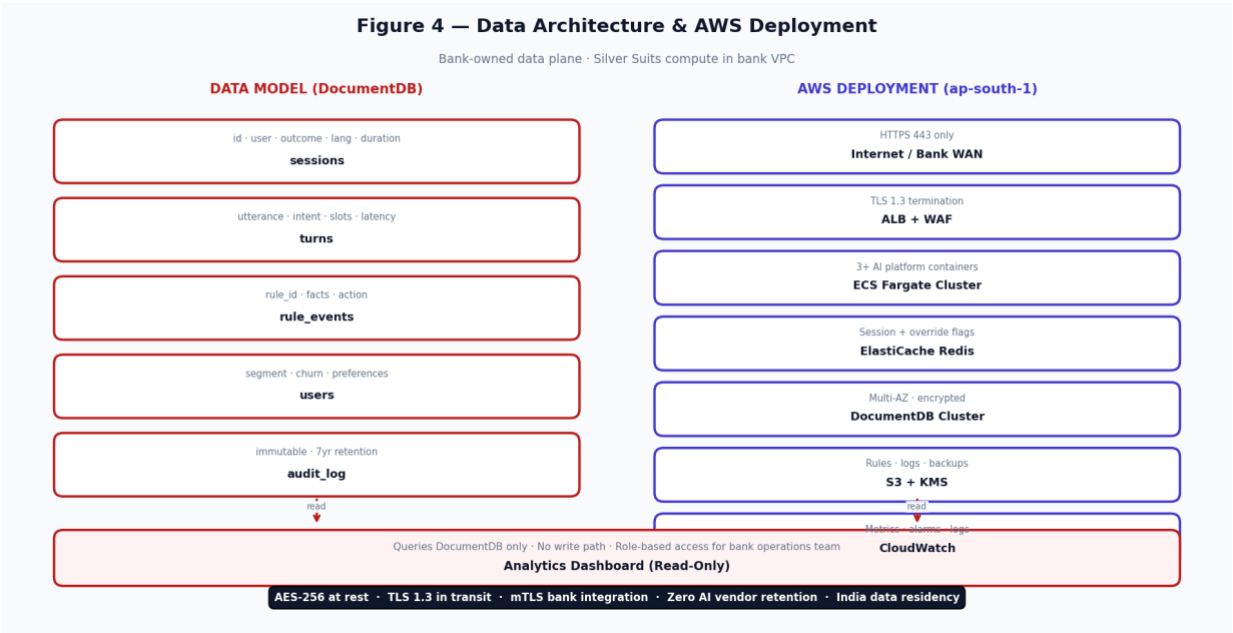


Figure 8 — DocumentDB collections + AWS ap-south-1 deployment topology

4.5 Demo vs Production — Component Matrix

Component	Demo (today)	Production (V1+)	Host
Backend	Express :3001	ECS Fargate + ALB + rule engine	Bank VPC Mumbai
STT	ElevenLabs proxy (dev keys)	ElevenLabs India residency	ElevenLabs India
LLM	OpenAI GPT-4o-mini	Azure OpenAI GPT-4o	Azure India
TTS	Cartesia proxy	Cartesia India region	Cartesia India
Banking data	mock.js in-memory	Bank REST via adapter	Bank CBS
Sessions	React state only	DocumentDB + Redis cache	Bank-owned
Rule engine	Client rage detect only	Full trigger → actuate pipeline	Silver Suits

5. Complete Data Workflow

5.1 Demo Data Flow (Current)

Step	Component	Data In	Data Out	Stored?
1	useRageDetect (browser)	pointerdown events, field validation errors	onFrustrated() callback fired	No — in-memory counters only
2	RMHelpPrompt	onFrustrated event	User taps "Yes, help me"	No

3	VoiceModal opens	Session object (blank)	INIT turn sent to server	No — session in React state
4	MediaRecorder + VAD	Microphone audio stream	Audio blob (webm/opus)	No
5	POST /api/stt	Audio blob + lang param	Transcript text + detected lang	No (ElevenLabs zero retention)
6	POST /api/engine/turn	Transcript + sessionId + lang	Updated session (state, history, pending)	No — server session map in-memory
7	LLM (OpenAI)	Utterance + state context + history (last 6)	intent, slots{}, reply, language	No (OpenAI zero retention)
8	Saga runner + tools	Resolved intent + slots	Tool results (contacts, payment)	No — mock in-memory
9	POST /api/tts	Bot reply text + lang	MP3 audio bytes (44100Hz, 128kbps)	No (Cartesia zero retention)
10	Browser Audio	MP3 blob URL	Audio played to user	No — blob URL revoked after play
11	MpinSheet (CONFIRM only)	User MPIN entry	Success → confirmation sent to server	No
12	Session result	DONE/FAILED/CANCELLED state	Result in session.history[] (React state only)	No

5.2 Production Data Flow — Pre-Conversation

Step	Action	Data Involved	Stored?	Version
1	Bank system emits event (button click, rage-click, inactivity)	event_type, user_id, form_profile{}, frontend_context{}	No	V1
2	Bank calls POST /api/trigger on Silver Suits backend	user_profile{}, event_type, form_profile{}, branch_context{}	No (rule evaluation only)	V1
3	Rule engine evaluates conditions in-memory (< 100ms)	All trigger payload tokens + loaded rule set	No	V1
4	Rule decision: action + script_id + language	action_type, script_id, lang, visibility	No	V1
5	Backend creates session record in bank DB	session_id, user_id, journey_type, lang, started_at	YES — bank DB	V1
5a	WebSocket push to browser → VoiceModal popup	script_id, lang, user_name, field_context	No (WebSocket ephemeral)	V1

5.3 Production Data Flow — During Conversation (Per Turn)

This is identical to the demo flow (steps 4–9 above) except AI provider keys use India data-residency configs, and step 7 additionally writes the turn to bank DB.

5.4 Production Data Flow — Post-Conversation

Step	Action	Data	Destination	Version
1	Session state → DONE/CANCELLED/FAILED	outcome, duration_s, turn_count	Bank DB — sessions collection (UPDATE)	V1
2	Optional result callback	session_id, outcome, transcript_summary	Bank backend (POST webhook)	V2

3	Post-session LLM analysis	Full transcript text	Azure OpenAI (India) — zero retention on LLM	V3
4	Structured analytics JSON stored	sentiment_arc, churn_risk, qa_score, script_compliance, flags[]	Bank DB — analytics collection	V3
5	Rule audit log written	rule_id, matched_params{}, action_taken, ts	Bank DB — rule_audit collection	V1

5.5 Data That Is NEVER Stored (Demo and Production)

- Voice audio recordings — discarded immediately after STT processing (zero retention on ElevenLabs)
- Voice biometric data — not collected at any stage of the pipeline
- Raw audio files — no disk writes on any AI service (STT, LLM, TTS)
- LLM conversation context — Azure OpenAI and OpenAI both configured with zero data retention
- Cartesia TTS inputs — not stored after audio generation
- Customer PII outside bank DB — Silver Suits backend never caches PII on disk (stateless)

6. Behavior Analysis — How Frustration Is Detected & What Triggers the AI

6.1 Current Implementation (IN DEMO CODEBASE — useRageDetect.js)

IN DEMO CODEBASE This is the actual behavior detection implemented in the code. It runs entirely client-side in the browser.

The useRageDetect hook monitors two independent frustration signals and fires a single onFrustrated() callback when either threshold is crossed. This opens the 'Need a little help?' bottom-sheet prompt. The hook auto-resets after 8 seconds so it can fire again if needed.

Signal	Threshold	Detection Logic	Code Location
Rage taps	5+ taps in 900ms on same element/screen	pointerdown timestamps pushed to array; array filtered to last 900ms; count >= RAGE_TAP_COUNT fires	useRageDetect.js — handleTap()
Repeated invalid field	2+ invalid submissions of same field	markInvalidField(fieldId) called on validation failure; counter per fieldId; >= INVALID_FIELD_THRESHOLD fires	useRageDetect.js — markInvalidField()

Constants (from code)

Constant	Value	Purpose
RAGE_TAP_COUNT	5	Number of taps needed within the window to trigger
RAGE_TAP_WINDOW	900ms	Sliding time window for tap counting
INVALID_FIELD_THRESHOLD	2	Times same field set invalid before triggering
Auto-reset delay	8000ms	Prevents re-fire for 8 seconds after triggering

What happens after frustration is detected (App.jsx)

- 1. onFrustrated() callback fires → setRmUpiPromptOpen(true)
- 2. RMHelpPrompt bottom-sheet appears: "Looks like you hit a snag. Want me to take over?"
- 3. User taps "Yes, help me" → setOpen(true) → VoiceModal opens → INIT turn sent to server
- 4. User taps "No, thanks" → dismiss() → rage detection resets → no AI session starts

6.2 Additional Trigger Points (IN DEMO CODEBASE)

Trigger	Where	How
HomeScreen AI button tap	HomeScreen.jsx	Mic button on home → handleMicTap() → VoiceModal opens with INIT turn
DemoPanel scripted prompt	DemoPanel.jsx	onSpeak(text) → runUtterance(text) → TRANSCRIPT turn with preset text
Quick action button	HomeScreen.jsx	onQuickAction(actionName) → handleOpenWithAction → START_ACTION turn
Navigate-to from AG-UI	homeAguiRunner.js	Home agent fires navigate_to tool → App.handleNavigate() → opens destination screen

6.3 Production Behavior Signals (Productization Roadmap)

In production, behavior signals come from both the client (same rage detection) and the bank backend system via the trigger API payload.

Signal	Source	How Sent to Silver Suits	Version
Rage taps (in-app)	Mobile app JS	Same useRageDetect hook + POST /api/trigger	V1
Inactivity duration	Mobile app timer	inactive_time_s in secondary_signals[]	V1
Form completion percentage	Mobile app	form_completion_pct in form_profile{}	V1
Repeat-click count	Mobile app	repeat_click_count in secondary_signals[]	V1
User profile / segment	Bank DB	user_profile{} in trigger payload	V1
Previous churn risk score	Bank analytics DB	user_profile.churn_risk_prev	V3
Session language preference	Bank user profile	user_profile.language_pref	V1

7. Rule Engine — Demo vs Production

In the demo, only client-side frustration detection opens the AI popup. In production (Figure 7), the bank sends events to POST /api/trigger and a full rule engine evaluates JSON rules in under 100ms.

7.1 What Exists in Demo Today (IN DEMO CODEBASE)

IN DEMO CODEBASE The demo has a routing rule set in homeAguiConfig.js (hardcoded) and a manifest/saga engine for conversation flows. No visual rule-builder exists yet.

Banking channel routing rules (homeAguiConfig.js — hardcoded)

Condition	Destination	Rationale
Amount > ₹1,00,000 OR IMPS/NEFT/bank account/IFSC mentioned	fund_transfer	Above UPI per-transaction limit of ₹1 lakh
Amount ≤ ₹1,00,000 OR UPI/mobile pay/phone mentioned	upi_payment	Within UPI limit
Loan request mentioned	loan_application	Loan LOS journey
Deposit/FD/RD/MMD/fixed deposit/recurring mentioned	create_deposit	Deposit creation journey
Transaction history/account statement/suspicious credit	transaction_history	Statement view journey
Completely ambiguous	Ask 1 clarifying question	Avoid wrong routing

Conversation state machine rules (server/engine/engine.js)

State	Allowed Inputs	Transition Logic
IDLE	INIT, START_ACTION, TRANSCRIPT	TRANSCRIPT → LLM extracts action → START_ACTION
FILL	TRANSCRIPT, CANCEL	TRANSCRIPT → extract slots → advance saga
ASK	TRANSCRIPT, CANCEL	TRANSCRIPT → fill slot → advance
DISAMBIGUATE	TRANSCRIPT, SELECTION, CANCEL	Pick from options list → advance
CHOOSE	TRANSCRIPT, SELECTION, CANCEL	Pick sub-option → advance
CONFIRM	CONFIRMATION, TRANSCRIPT (yes/no), CANCEL	yes → execute tool_call; no → CANCELLED
DONE	RESET, START_ACTION	Terminal — auto-reset on next action
FAILED	RESET, START_ACTION	Terminal — error surfaced to user
CANCELLED	RESET, START_ACTION	Terminal — transcript saved as-is

7.2 Productization Rule Engine (V1+ — New Build)

IMPORTANT The production rule engine is a new build on top of the same backend, evaluating bank-provided trigger payloads rather than in-app routing.

Production trigger payload — what the bank sends to POST /api/trigger:

Field	Type	Description	Required
user_id	string	Bank's internal user identifier	Yes
user_profile	object	name, segment, language_pref, account_type, churn_risk_prev	Yes
event_type	enum	help_button_clicked / rage_click / inactivity / account_created / custom	Yes

form_profile	object	field_ids[], field_statuses{}, validation_regex{}, form_completion_pct	Conditional
frontend_context	object	current_page, journey_type, button_id, timestamp_ms	Yes
branch_context	object	branch_code, nearest_human_rm_id, geography	Optional
secondary_signals	array	rage_click_count, inactivity_duration_s, repeat_click_count	Optional

Example production rules:

```
IF [form_completion_pct > 75] AND [repeat_click_count > 5]
  THEN [action=trigger_popup, script=loan_help, lang=en]

IF [inactive_time_s > 20] AND [event_type = inactivity]
  THEN [action=trigger_popup, script=push_user, lang=hi]

IF [churn_risk_prev > 70] AND [event_type = rage_click]
  THEN [action=assign_human_rm, script=retention_assist, priority=high]
```

7.3 Actions the Rule Engine Can Actuate (Production)

Action	How Actuated	Parameters	Version
trigger_popup	WebSocket push to browser	script_id, lang, user_name, field_context	V1
assign_human_rm	POST bank's CRM webhook	user_id, branch_code, rm_id, priority	V1
send_notification	Bank notification API (SMS/push)	user_id, template_id, params{}	V2
fill_form_field	WebSocket command to browser	field_id, value, validation_regex	V1
log_event	Write to bank DB	session_id, event_type, payload, ts	V1
abort_session	WebSocket close + state = CANCELLED	reason_code, user_id	V1

8. AG-UI Streaming Agents — Per-Screen AI (Deep Dive)

8.1 What AG-UI Is (IN DEMO CODEBASE)

AG-UI (Agent-UI) is a Server-Sent Events (SSE) protocol where the backend streams AI agent actions to the browser in real time. Each screen has its own dedicated agent running OpenAI function calling. The agent can call tools (e.g. set_field) that update the form live while the user is speaking — no waiting for a full response before the form starts filling.

8.2 Per-Agent Specification

Home Screen Agent (`indian_bank_home_assistant`)

File: `server/agui/homeAguiRunner.js` + `homeAguiConfig.js`

Role: Acts as a smart banking concierge. Understands what the customer wants (payment, loan, deposit, statement) and routes them to the correct screen using the `navigate_to` tool.

Tools

`navigate_to(destination: upi_payment|fund_transfer|loan_application|create_deposit|transaction_history, context: string, routing_status: string)`

Logic / Flow

- Amount > ₹1L OR IMPS/NEFT → `fund_transfer`
- Amount ≤ ₹1L OR UPI → `upi_payment`
- Loan request → `loan_application`
- Deposit/FD/RD → `create_deposit`
- Transaction query → `transaction_history`

Loan LOS Agent (`indian_bank_loan_los`)

File: `server/agui/loanAguiRunner.js` + `loanAgentConfig.js` + `loanTools.js`

Role: Fills the loan application form in real time as the user speaks. Can infer multiple field values from one utterance and fill them simultaneously.

Tools

`set_field(fieldId: occupation|subProduct|purposeLoan|variant|facility|proposal|interestType|loanAmount|tenureMonths|branchPin, value: string)`

`validate_form(state: current form state — returns validation snapshot)`

Logic / Flow

- 10 loan fields
- Select fields: enum-validated (o0/o1/o2, lp1-lp5, etc.)
- Numeric: `loanAmount`, `tenureMonths`
- Text: `branchPin`

IMPS Fund Transfer Agent (`indian_bankimps_transfer`)

File: `server/agui/impsAguiRunner.js` + `impsAguiConfig.js` + `impsTools.js`

Role: Guides the user through within-bank or other-bank IMPS transfer form.

Tools

`set_field(fieldId: transferType|payeeType|payeeName|payeeAccountNo|ifsc|payeeBank|mobileNo|amount|remarks)`

Logic / Flow

- Asks ONE question at a time (strict flow)
- within-bank: name + account number
- other-bank/account: IFSC + account
- other-bank/mobile: bank name + mobile number

Create Deposit Agent (indian_bank_create_deposit)

File: server/agui/depositAguiRunner.js + depositAguiConfig.js

Role: Helps user select deposit type (FD/RD/MMD) and fill amount, tenure, maturity instructions.

Tools

set_field(fieldId: various deposit form fields)

Logic / Flow

- FD: lump sum, fixed tenure
- RD: monthly installment, tenure
- MMD: money multiplier deposit

Transaction History Agent (indian_bank_txn_history)

File: server/agui/txnHistoryAguiRunner.js + txnHistoryAguiConfig.js

Role: Answers user questions about their account statement, suspicious transactions, or expected credits. Can navigate to payment flows for relevant queries.

Tools

set_field(display/filter fields)

navigate_to(when user wants to initiate a payment based on a transaction)

Logic / Flow

- Suspicious credit / fraud alert → explain, log
- Wanted to pay someone → navigate_to upi_payment

8.3 SSE Streaming Protocol

All AG-UI agents stream via Server-Sent Events. Each SSE event is a JSON object with a type field. The browser processes events in order to update UI state in real time.

SSE Event Type	Payload	Browser Effect
TEXT_MESSAGE_START	{messageId, role}	New message bubble opened
TEXT_MESSAGE_CONTENT	{messageId, delta}	Text appended to bubble (streaming)
TEXT_MESSAGE_END	{messageId}	Bubble finalised; TTS triggered on text
TOOL_CALL_START	{toolCallId, toolCallName}	Agent about to call a tool
TOOL_CALL_ARGS	{toolCallId, delta}	Tool args streamed

TOOL_CALL_END	{toolCallId}	Tool call complete
RUN_FINISHED	{threadId, runId}	Agent turn complete
RUN_ERROR	{message, code}	Error shown to user

9. Voice Bot Interaction Flow & Per-Turn Logging

9.1 Per-Turn Pipeline (IN DEMO CODEBASE)

Step	Component	Action	Target Latency
1	Browser (in-app)	User speaks; MediaRecorder starts on mic permission grant	—
2	AnalyserNode (VAD)	RMS checked every animation frame; 1.5s silence-stop armed after speech starts	—
3	MediaRecorder.stop()	chunks[] → Blob (webm/opus); recording ends	—
4	POST /api/stt?lang=en	Audio blob → ElevenLabs Scribe v2 → transcript text + lang code	300–500ms
5	POST /api/engine/turn	Transcript + sessionId → server state machine processes turn	—
6	LLM extract() (OpenAI)	utterance + state context + last 6 history turns → intent JSON	400–700ms
7	applyIntent() + advance()	Slots merged; saga stepped forward; next pending computed	< 20ms
8	Tool calls (if needed)	contacts.search / billers.search / execute_payment (mock)	50–200ms (mock)
9	Bot reply text resolved	Session updated; history[] and state returned to client	—
10	POST /api/tts	Bot reply text → Cartesia sonic-3 → MP3 bytes returned	300–500ms
11	Audio.play()	MP3 blob URL played in browser; TTS flag set	—
12	waitUntilTtsIdle()	Auto-mic arms 500ms after TTS ends; loop back to step 1	—

9.2 Hands-Free Auto-Mic Logic (App.jsx)

The auto-mic effect re-arms the microphone after each bot reply so the user never has to tap the mic button between turns. It is gated by multiple conditions to prevent the mic from capturing TTS audio or opening while the state machine is thinking.

Condition Checked	Why
open === true	Modal must be visible
mpinOpen === false	MPIN sheet takes priority — user is entering their PIN
speech.supported === true	STT backend available
session.thinking === false	LLM is not currently processing
session.executing === false	Tool call is not in flight

session.state in VOICE_INPUT_STATES	Only arm in FILL / DISAMBIGUATE / CHOOSE / CONFIRM states
ttsPlaying === false	Bot is not currently speaking
waitUntilTtsIdle() resolved	Belt-and-suspenders TTS idle check
500ms pause after TTS ends	Let user absorb the bot message before mic opens

9.3 MPIN Authentication Gate (MpinSheet.jsx + App.jsx)

Every payment confirmation is gated by MPIN authentication. The voice "yes" interceptor in App.jsx catches voice confirmations and routes them through the MPIN sheet before the actual confirmation is sent to the server.

Scenario	Flow
User taps "Confirm" button	handleConfirm() → setMpinOpen(true) → MpinSheet shown
User speaks "yes"/"haan" during CONFIRM state	looksLikeYes() → speech.abort() → setMpinOpen(true)
User speaks "no"/"cancel"	Passes through normally → handleConfirmTap(session, false)
MPIN success	setMpinOpen(false) → handleConfirmTap(session, true) → server executes payment
MPIN cancel	setMpinOpen(false) → stay on CONFIRM screen → user can re-tap Confirm

9.4 What Is Logged Per Turn (session.history[] — Demo)

Field	Type	Description
role	enum: user bot	Who said it
text	string	Exact STT transcript or LLM-generated bot reply
t	unix timestamp ms	Date.now() at time of push to history

IMPORTANT In the demo, session.history[] lives in server-side in-memory Map only. Nothing is written to a database. In production (V1+), every turn is INSERTed into bank DB (turns collection) with additional fields: intent, slots, engine_state, lang.

9.5 Production Session Lifecycle Events (V1+ — Written to Bank DB)

Event	Trigger	Data Written to Bank DB
SESSION_STARTED	First turn processed	session_id, user_id, journey_type, lang, channel, trigger_event, consent_given, ts
TURN_LOGGED	After every conversation turn	session_id, role, text, intent, slots{}, engine_state, lang, t
FORM_FIELD_FILLED	fill_form_field tool succeeds	session_id, field_id, value, validated, ts
CONFIRMATION_GIVEN	User confirms (post-MPIN)	session_id, action_summary, slots_snapshot, ts
FALLBACK_TRIGGERED	Exception path entered	session_id, fallback_type, reason, ts
CONSENT_RECORDED	User accepts/declines session start	session_id, consent_given: bool, channel, ts
SESSION_ENDED	State → DONE/CANCELLED/FAILED	session_id, outcome, duration_s, turn_count, final_state, ended_at

HUMAN_RM_ASSIGNED	Escalation rule fires	session_id, user_id, rm_id, branch_code, reason, ts
-------------------	-----------------------	---

10. Bank DB Integration — Read / Write Specification (Productization V1+)

IMPORTANT None of this exists in the demo. All data in demo is mock/in-memory. This section describes the V1+ production integration.

10.1 Database Schema (DocumentDB / MongoDB)

Collection	Key Fields	Indexes	Retention
sessions	session_id, user_id, journey_type, outcome, started_at, ended_at, lang, turn_count, channel	user_id + started_at; outcome + started_at	Per bank policy (≥7 yrs financial per RBI)
turns	session_id, role, text, intent, slots{}, engine_state, lang, t	session_id + t	Same as sessions
form_events	session_id, field_id, value, validated, ts	session_id + field_id	Same as sessions
analytics	session_id, user_id, sentiment_arc[], churn_risk, qa_score, script_compliance, talk_ratio, flags[]	session_id; user_id + churn_risk	Same as sessions
rule_audit	rule_id, version, matched_params{}, action_taken, ts	ts; rule_id	2 years (compliance audit)
escalations	session_id, user_id, rm_id, branch_code, reason, ts	user_id + ts	2 years

10.2 Write Operations

Operation	Trigger	Collection	Payload Size	Version
INSERT session	Session starts	sessions	~500 B	V1
INSERT turn (per turn)	After each dialogue turn	turns	200–800 B	V1
INSERT form_event	Form field filled	form_events	~200 B	V1
UPDATE session outcome	Session ends	sessions	Patch only	V1
INSERT analytics	Post-session LLM analysis (V3)	analytics	2–5 KB	V3
INSERT rule_audit	Rule match fires	rule_audit	~300 B	V1
INSERT escalation	Human RM assigned	escalations	~200 B	V2

10.3 Read Operations

Operation	When	Purpose	Collection
GET user profile	Pre-session (rule	Contextualise script + language	Bank's user table (read-only view)

	engine)	selection	
GET previous sessions (user)	Pre-session (V3 churn)	Feed prior outcomes into rule engine	sessions
GET session + turns	Dashboard: timeline	Per-user history + conversation replay	sessions + turns
GET analytics record	Dashboard: QA/sentiment	Display analytics per session	analytics
GET aggregate by date	Dashboard: volume charts	Session count / outcome rate over time	sessions
GET escalations by user	Rule engine cap check	Count recent escalations for abuse guard	escalations

12. Resource Requirements

12.1 Bank-Side Integration Effort

Task	Bank Role	Effort	When
Wire rage-detect + trigger API in mobile app	Mobile/Backend engineer	1–2 weeks	V1 — Wk 2
Provision DocumentDB schema + access grants	DBA	1 week	V1 — Wk 2
Whitelist Silver Suits IPs on DB + API firewall	Security / DBA	1–2 days	V1 — Wk 2
Implement result callback receiver	Backend engineer	3–5 days	V2 — Wk 10
Connect human RM escalation webhook	CRM / Backend	3–5 days	V2 — Wk 11
Provide branding guidelines	Marketing / Design	1 week	V2 — Wk 11
Compliance and legal review	Compliance / Legal	2–3 weeks	V1 — Wk 1–3

13. Timeline & Version Feature List

Version	Weeks	Theme	Key Milestone
V1 — MVP	1–8	Bank integration + real DB	Trigger API live, real sessions persist to bank DB
V2 — Production-Ready	9–20	Rule builder + analytics v1	Rule-builder UI, analytics dashboard, push notifications

13.1 Sprint Plan

Sprint	Ver	Deliverables
1–2	V1	Azure backend, bank API contract signed, DB schema v1, CI/CD, TLS/auth setup
3–4	V1	Production rule engine (IF/THEN), POST /api/trigger endpoint, real STT/TTS keys (India residency)
5–6	V1	Bank DB writes (sessions, turns, form_events), consent flow, cap/rate limiting
7–8	V1	Exception handling (all paths), human RM webhook, V1 UAT with bank team
9–10	V2	Push notification gateway (FCM/APNs), WebSocket session channel hardening
11–12	V2	Rule-builder UI (visual editor → JSON config), 6 additional languages
13–14	V2	Emergency override dashboard, DEV/PROD split, eval harness
15–16	V2	Result callback API, branding customisation, V2 load testing (1000 concurrent)
17–18	V2	Compliance audit, DPA/NDA execution, VAPT, V2 go-live
18–19	V3	Full analytics (multi-journey, timeline, conversation replay)
19–20	V3	Revenue-opportunity extraction, predictive triggers from churn score, V3 go-live

13.2 V1 Feature List — Weeks 1–8 (Building on Demo)

Feature	Demo Status	V1 Action
STT (ElevenLabs Scribe v2)	IN DEMO (dev keys)	Switch to India data residency + production keys
LLM (OpenAI GPT-4o-mini)	IN DEMO (dev keys)	Switch to Azure OpenAI India endpoint
TTS (Cartesia sonic-3)	IN DEMO (dev keys)	Switch to Cartesia India region + production keys
Manifest/saga state machine	IN DEMO (complete)	No change needed — add new manifests as required
AG-UI screen agents (all 5)	IN DEMO (complete)	No change needed for V1
useRageDetect behavior detection	IN DEMO (complete)	Bank embeds in their mobile app SDK
Trigger REST API	NOT IN DEMO	Build POST /api/trigger with rule engine evaluation
Production rule engine	NOT IN DEMO	Build in-memory expression evaluator, JSON rule configs
Bank DB writes (sessions + turns)	NOT IN DEMO (mock only)	Wire DocumentDB driver; write on every turn
Consent management	NOT IN DEMO	Add consent gate before first session turn
Per-user cap / rate limiting	NOT IN DEMO	Read escalations count from DB; abort at cap
Result callback API	NOT IN DEMO	POST outcome + transcript summary to bank after session

14. Compliance Quick-Reference

Component	Host	Data Stored?	AI Training?	India?	Cert
Backend API	Azure India (Central + South)	No (stateless)	N/A	Yes	SOC2/ISO27001/PCI DSS
STT (ElevenLabs)	ElevenLabs India data residency	Zero retention	No	Yes	SOC2 Type II, HIPAA
LLM (Azure)	Azure India — Chennai	Zero retention	No	Yes	SOC2 Type II, ISO27001

OpenAI)	endpoint				
TTS (Cartesia)	Cartesia India region	Zero retention	No	Yes	SOC2 Type II
Bank Database	Bank-controlled AWS Mumbai / OnPrem	Permanent — bank owns	N/A	Yes	AWS SOC2 / bank cert
Analytics Dashboard	Azure India	No (reads bank DB)	N/A	Yes	SOC2/ISO27001

Encryption: AES-256 at rest, TLS 1.2+ in transit — mandatory across all components. DPDP Act 2023 + RBI guidelines compliance. 7-year minimum retention for financial records (PMLA). Bank owns all persistent data. Silver Suits never retains voice audio or customer PII on disk.

15. Exception Handling & Fallbacks

15.1 Demo — Currently Handled (IN DEMO CODEBASE)

Exception	How Handled (code)	File
STT returns empty transcript	engine re-prompts with "didnt_catch"; previousBotMessage re-spoken	engine.js — handleSmalltalk()
LLM unavailable / API error	Speaks "model unavailable" message; does not loop; turn ends gracefully	engine.js — intent.error === llm_unavailable
Unknown intent (no action match)	handleSmalltalk() — speaks LLM contextual reply OR re-prompts pending slot	engine.js — handleSmalltalk()
User says cancel/nahi/stop	State → CANCELLED; cancellation message spoken	engine.js — doCancel()
Bank mock API returns error	State → FAILED; BackendError code shown to user; retryable flag	backend.js — BackendError class
Bank mock: random 5–7% failure	Simulates real bank decline; user sees FAILED with retry option	backend.js — flaky()
Force-fail mode (demo panel)	bank_declined / network_error injected via X-Force-Fail header	backend.js + DemoPanel.jsx
Disambiguation (multiple matches)	State → DISAMBIGUATE; options list shown; user picks by voice or tap	saga.js + VoiceModal.jsx
TTS 503 (Cartesia not configured)	Console warning only; conversation continues without TTS audio	cartesiaTts.js

15.2 Production — Additional Fallbacks (V2+)

Exception	Detection	Action	Logged?
Tech glitch / connection failure	WebSocket or API error event	Log; assign to nearest human RM	Yes
Session abandoned	User closes app mid-session	Log; 2 abandons/day → human RM	Yes
Session dropped midway	WebSocket close / network loss	Save transcript; log as incomplete	Yes
Insufficient rule parameters	Rule eval fails validation	Use default action; flag to team	Yes

User exceeds per-session cap	Cap counter in DB	Abort; log; assign human RM	Yes
------------------------------	-------------------	-----------------------------	-----

16. Non-Functional Requirements

Requirement	Demo (Current)	Production Target	Notes
E2E turn latency	~1.0–1.5s (STT+LLM+TTS)	< 1.2s in-app	STT 500ms + LLM 600ms + TTS 400ms (parallel where possible)
Rule engine decision	N/A (instant in-browser routing)	< 100ms	In-memory eval; no DB calls during evaluation
Concurrent sessions	~10 (dev server)	1,000+ simultaneous	Azure App Service auto-scale; load tested in V2 (week 17–18)
System availability	N/A (local dev)	99.9% uptime	Azure + ElevenLabs + Cartesia SLAs combined
Data freshness (dashboard)	N/A	< 5 min lag	Async DB writes; dashboard reads directly from bank DB
Rule change cycle	Instant (hardcoded)	14 business days avg (prod rules)	Emergency override bypasses for urgent compliance updates
VAPT / Security	Not applicable (mock)	Before each major go-live	Penetration testing before V2 and V3 go-live
RBI/DPDP audit	N/A	Continuous readiness	30-day purge cycle; full audit logs in bank DB

This document is confidential and intended for Indian Bank's technical and product teams only.

Silver Suits AI · Demo + Production architecture + Productization roadmap — May 2026